

LEC 2

أسئلة

MCQ – Deep Feedforward Networks (30 Questions)

1. What does the function $\text{ReLU}(x)$ compute?

- A) $\min(0, x)$
- B) $\max(0, x)$
- C) x^2
- D) $\text{sigmoid}(x)$

 **Answer:** B

 **Explanation:** ReLU outputs zero for negative inputs and keeps positive values.

2. The function $f = \text{ReLU}(Ax)$ is considered:

- A) Nonlinear
- B) Linear
- C) Convex only
- D) Recurrent

 **Answer:** A

 **Explanation:** The ReLU activation makes the function nonlinear.

3. Why is stacking ReLU layers important?

- A) To reduce parameters
- B) To make the model linear
- C) To increase nonlinearity
- D) To remove activation functions

 **Answer:** C

 **Explanation:** Multiple nonlinear layers increase model expressiveness (deep vs shallow).

4. Moving from shallow to deep networks means:

- A) Fewer layers
- B) Linear to nonlinear mappings

- C) Removing ReLU
- D) Using convex optimization

 **Answer:** B

 **Explanation:** Depth allows modeling complex nonlinear functions.

5. The expression $y = \text{ReLU}(B \text{ReLU}(Ax))$ represents:

- A) A linear model
- B) A shallow network
- C) A deep feedforward network
- D) A recurrent network

 **Answer:** C

 **Explanation:** It is a composition of multiple nonlinear transformations.

6. Why can XOR not be solved by a linear model?

- A) Too many inputs
- B) Not convex
- C) Not linearly separable
- D) Too much noise

 **Answer:** C

 **Explanation:** XOR requires nonlinear decision boundaries.

7. A Multi-Layer Perceptron (MLP) is:

- A) A recurrent network
- B) A feedforward neural network
- C) A probabilistic model
- D) A kernel method

 **Answer:** B

 **Explanation:** MLPs are feedforward networks without feedback connections.

8. The goal of a feedforward neural network is to:

- A) Memorize data
- B) Approximate a function f^*
- C) Solve convex problems only
- D) Remove noise

✓ **Answer:** B

📝 **Explanation:** Neural networks aim to approximate an unknown target function.

9. Feedforward networks are called “feedforward” because:

- A) They use gradients
- B) They have cycles
- C) Information flows only forward
- D) They update weights backward

✓ **Answer:** C

📝 **Explanation:** Data flows from input to output without feedback loops.

10. Networks with feedback connections are called:

- A) CNNs
- B) MLPs
- C) Feedforward networks
- D) Recurrent neural networks

✓ **Answer:** D

📝 **Explanation:** Feedback connections define recurrent architectures.

11. A deep feedforward network can be written as:

- A) $f(x) = Ax$
- B) $f(x) = h_1(x)$
- C) $f(x) = h_1 \circ h_{l-1} \circ \dots \circ h_1(x)$
- D) $f(x) = \text{sigmoid}(x)$

✓ **Answer:** C

📝 **Explanation:** Deep networks are compositions of multiple functions.

12. Each h_l in a deep network represents:

- A) A dataset
- B) A loss function
- C) A layer/module
- D) A gradient

✓ **Answer:** C

📝 **Explanation:** Each h_l is a transformation (layer).

13. A neural network can be visualized as:

- A) A tree
- B) A cyclic graph
- C) A directed acyclic graph
- D) An undirected graph

 **Answer:** C

 **Explanation:** Feedforward networks form DAGs.

14. A module in a neural network is:

- A) Only a weight
- B) A loss function
- C) A building block / transformation
- D) A dataset

 **Answer:** C

 **Explanation:** Modules are functions that transform inputs to outputs.

15. A module may:

- A) Never have parameters
- B) Always be linear
- C) Have trainable parameters or not
- D) Only work on scalars

 **Answer:** C

 **Explanation:** Some modules have parameters, others don't.

16. Which of the following is a valid module?

- A) $f(x) = Ax$
- B) $f(x) = \exp(x)$
- C) Both A and B
- D) Neither

 **Answer:** C

 **Explanation:** Both are transformations (modules).

17. ReLU is typically applied:

- A) To the entire vector at once
- B) Element-wise

- C) Only to scalars
- D) Only at the output

✅ **Answer:** B

📝 **Explanation:** ReLU is applied element-wise to activations.

18. In modern neural networks, the default activation is often:

- A) Sigmoid
- B) Tanh
- C) ReLU
- D) Softmax

✅ **Answer:** C

📝 **Explanation:** ReLU is preferred for efficiency and gradient flow.

19. The function

$f(x; W, c, w, b) = w^T \max(0, Wx + c) + b$
represents:

- A) A linear classifier
- B) A deep CNN
- C) A one-hidden-layer MLP
- D) A kernel machine

✅ **Answer:** C

📝 **Explanation:** It is an MLP with one hidden layer and ReLU activation.

20. Why do we need bias terms?

- A) To reduce parameters
- B) To shift activation functions
- C) To remove nonlinearity
- D) To normalize data

✅ **Answer:** B

📝 **Explanation:** Bias allows flexible shifting of activations.

21. Activation functions must be:

- A) Continuous only
- B) Second-order differentiable everywhere
- C) First-order differentiable almost everywhere
- D) Linear

✓ **Answer:** C

📝 **Explanation:** Required for gradient-based optimization.

22. Special care is needed when networks contain:

- A) ReLU units
- B) Biases
- C) Cycles
- D) Linear layers

✓ **Answer:** C

📝 **Explanation:** Cycles complicate gradient computation.

23. Most CNNs and Transformers are:

- A) Recurrent models
- B) Feedforward models
- C) Probabilistic models
- D) Linear models

✓ **Answer:** B

📝 **Explanation:** They are primarily feedforward architectures.

24. Linear models are limited because they:

- A) Are slow
- B) Are non-convex
- C) Have limited capacity
- D) Cannot be trained

✓ **Answer:** C

📝 **Explanation:** They cannot represent complex nonlinear patterns.

25. A common way to extend linear models is to:

- A) Remove bias
- B) Add noise
- C) Transform the input $\phi(x)$
- D) Use random labels

✓ **Answer:** C

📝 **Explanation:** Nonlinear feature transformation increases capacity.

26. In deep learning, $\phi(x;\theta)$ is:

- A) Fixed
- B) Learned
- C) Always linear
- D) A kernel function

 **Answer: B**

 **Explanation:** Deep networks learn the feature transformation.

27. Training deep models is generally:

- A) Convex
- B) Closed-form
- C) Non-convex
- D) Deterministic

 **Answer: C**

 **Explanation:** Learning θ makes the problem non-convex.

28. A “good” representation means:

- A) High-dimensional
- B) Sparse
- C) Linearly separable
- D) Noisy

 **Answer: C**

 **Explanation:** Linear models can solve the task in that space.

29. XOR becomes solvable by a linear model:

- A) In original input space
- B) After normalization
- C) In the learned feature space
- D) Never

 **Answer: C**

 **Explanation:** Neural networks transform inputs into separable features.

30. The key idea of deep learning is:

- A) Designing the exact function
- B) Learning hierarchical representations

- C) Using convex optimization
- D) Removing nonlinearities

✅ **Answer:** B

📝 **Explanation:** Deep learning learns representations through multiple layers.

MCQ – Deep Learning (Hierarchies, Optimization & Activations)

1. Mixing hierarchies in neural networks makes sense when:

- A) The model is shallow
- B) We have no domain knowledge
- C) We have good problem-domain knowledge
- D) The data is unlabeled

✅ **Answer:** C

📝 **Explanation:** Domain knowledge helps design effective hierarchies.

2. Mixing hierarchies is especially useful when:

- A) Using single input
- B) Combining multiple modalities
- C) Using linear models
- D) Training small datasets

✅ **Answer:** B

📝 **Explanation:** Different inputs (e.g., RGB & LiDAR) benefit from separate hierarchies.

3. Combining images and text is an example of:

- A) Linear regression
- B) Single-modality learning
- C) Multi-modal learning
- D) Reinforcement learning

✅ **Answer:** C

📝 **Explanation:** It uses multiple data modalities.

4. Directed Acyclic Graph (DAG) models:

- A) Contain cycles
- B) Allow feedback loops

- C) Have no cycles
- D) Cannot be trained

 **Answer:** C

 **Explanation:** DAGs do not contain cycles.

5. Skip connections are used to:

- A) Increase vanishing gradients
- B) Remove nonlinearity
- C) Improve information flow
- D) Reduce parameters to zero

 **Answer:** C

 **Explanation:** Skip connections help gradient propagation.

6. Interweaved connections mainly help:

- A) Overfitting
- B) Training instability
- C) Feature reuse
- D) Removing hierarchy

 **Answer:** C

 **Explanation:** They allow reuse of intermediate features.

7. A key trade-off in model design is between:

- A) Accuracy and bias
- B) Complexity and efficiency
- C) Memory and labels
- D) Depth and loss

 **Answer:** B

 **Explanation:** More complex models are usually less efficient.

8. Often, better performance is achieved by:

- A) Few iterations with a strong model
- B) Many iterations with a weaker model
- C) No training
- D) Only linear models

✓ **Answer: B**

📝 **Explanation:** More training can outperform stronger but under-trained models.

9. ReLUs are considered “half-linear” because:

- A) They are quadratic
- B) They saturate on both sides
- C) They are linear only for $x > 0$
- D) They are probabilistic

✓ **Answer: C**

📝 **Explanation:** ReLU is linear for positive inputs only.

10. The main practical constraint in large models is often:

- A) Dataset size
- B) CPU speed
- C) GPU memory
- D) Activation choice

✓ **Answer: C**

📝 **Explanation:** GPU memory limits model size.

11. A hierarchy of modules means:

- A) Independent layers
- B) Outputs feed future inputs
- C) No parameter sharing
- D) Only linear transformations

✓ **Answer: B**

📝 **Explanation:** Each module’s output is the next module’s input.

12. Loopy connections require:

- A) Removal
- B) Linearization
- C) Unfolding the graph
- D) No training

✓ **Answer: C**

📝 **Explanation:** Cycles must be unfolded (RNNs).

13. Loopy connections are mostly:

- A) Preferred today
- B) Not used anymore
- C) Convex
- D) Linear

 **Answer: B**

 **Explanation:** RNN-style loops are less common now.

14. Nonlinearity causes the loss function to be:

- A) Convex
- B) Linear
- C) Non-convex
- D) Closed-form

 **Answer: C**

 **Explanation:** Nonlinear activations make optimization non-convex.

15. Because the loss is non-convex, we need:

- A) Exact solvers
- B) Linear algebra
- C) Iterative gradient-based optimizers
- D) Closed-form solutions

 **Answer: C**

 **Explanation:** SGD is commonly used.

16. Gradient-based learning is sensitive to:

- A) Dataset labels
- B) Initialization
- C) Batch size only
- D) Loss type only

 **Answer: B**

 **Explanation:** Bad initialization can hurt convergence.

17. To update weights using gradients, we need:

- A) Activation function
- B) Cost (loss) function

- C) Dataset only
- D) Optimizer only

 **Answer: B**

 **Explanation:** Loss measures how good the model is.

18. Maximum Likelihood Estimation (MLE) aims to:

- A) Minimize weights
- B) Maximize $p(y|x; w)$
- C) Minimize gradients
- D) Linearize the model

 **Answer: B**

 **Explanation:** MLE finds parameters that best explain data.

19. Minimizing negative log-likelihood is equivalent to:

- A) Maximizing loss
- B) Minimizing cross-entropy
- C) Maximizing accuracy
- D) Removing noise

 **Answer: B**

 **Explanation:** Cross-entropy comes from NLL.

20. If $p(y|x)$ is Gaussian, the cost function becomes:

- A) Cross-entropy
- B) KL-divergence
- C) Mean Squared Error
- D) Hinge loss

 **Answer: C**

 **Explanation:** Gaussian likelihood leads to MSE.

21. Euclidean (L2) loss is mainly used for:

- A) Classification
- B) Clustering
- C) Regression
- D) Ranking

✓ **Answer:** C

📝 **Explanation:** It fits continuous targets.

22. Euclidean loss is sensitive to outliers because:

- A) It ignores large errors
- B) Errors grow linearly
- C) Errors grow quadratically
- D) It is non-differentiable

✓ **Answer:** C

📝 **Explanation:** Squared errors magnify large deviations.

23. A good cost function should have gradients that are:

- A) Zero
- B) Random
- C) Large and predictable
- D) Always constant

✓ **Answer:** C

📝 **Explanation:** Helps guide learning effectively.

24. Activation functions that saturate cause:

- A) Faster training
- B) Large gradients
- C) Vanishing gradients
- D) Convex loss

✓ **Answer:** C

📝 **Explanation:** Flat regions reduce gradient magnitude.

25. Linear activation functions:

- A) Saturate
- B) Have stable gradients
- C) Are non-differentiable
- D) Cause dead neurons

✓ **Answer:** B

📝 **Explanation:** Gradients remain strong and stable.

26. The ReLU activation is defined as:

- A) $\min(0, x)$
- B) x^2
- C) $\max(0, x)$
- D) $\exp(x)$

 **Answer:** C

 **Explanation:** Outputs zero for negative inputs.

27. One advantage of ReLU is:

- A) Zero-centered output
- B) Sparse activation
- C) Bounded output
- D) Smooth everywhere

 **Answer:** B

 **Explanation:** About 50% neurons are inactive.

28. A potential problem of ReLU is:

- A) Vanishing gradients everywhere
- B) High computational cost
- C) Dead neurons
- D) Non-convex loss

 **Answer:** C

 **Explanation:** Neurons may stop activating completely.

29. Leaky ReLU addresses dead neurons by:

- A) Using $\exp(x)$
- B) Allowing small gradient for $x \leq 0$
- C) Clipping outputs
- D) Removing nonlinearity

 **Answer:** B

 **Explanation:** Keeps gradients alive for negative inputs.

30. ELU activation differs from ReLU because it:

- A) Is linear everywhere
- B) Is discontinuous

- C) Is smooth for $x \leq 0$
- D) Has zero gradient

✅ **Answer:** C

📝 **Explanation:** ELU provides smooth negative outputs.

🧠 MCQ – Activations, Architecture, Depth & Optimization (30 Questions)

1. GELU stands for:

- A) Generalized Linear Unit
- B) Gaussian Error Linear Unit
- C) Gradient Enhanced Linear Unit
- D) Global Exponential Linear Unit

✅ **Answer:** B

📝 **Explanation:** GELU = Gaussian Error Linear Unit.

2. GELU is similar to:

- A) ReLU only
- B) Sigmoid
- C) ELU
- D) Softmax

✅ **Answer:** C

📝 **Explanation:** GELU is similar to ELU but non-monotonic.

3. GELU is non-monotonic, meaning:

- A) Output is always positive
- B) Gradient changes sign
- C) It saturates
- D) It is linear

✅ **Answer:** B

📝 **Explanation:** Non-monotonic means the gradient can change sign.

4. GELU is the default activation for:

- A) CNNs
- B) RNNs

- C) Vision Transformers
- D) Autoencoders

 **Answer:** C

 **Explanation:** GELU is standard in Vision Transformers.

5. Compared to sigmoid, tanh has output range:

- A) $[0, 1]$
- B) $[-1, 1]$
- C) $(-\infty, \infty)$
- D) $[0, \infty)$

 **Answer:** B

 **Explanation:** tanh outputs values in $[-1, 1]$.

6. One advantage of tanh over sigmoid is:

- A) No saturation
- B) Zero-centered output
- C) Lower computation
- D) Convex loss

 **Answer:** B

 **Explanation:** tanh outputs are centered around zero.

7. Zero-centered activations help by:

- A) Increasing bias
- B) Weakening gradients
- C) Improving gradient flow
- D) Removing nonlinearity

 **Answer:** C

 **Explanation:** Zero-mean activations reduce bias shift.

8. A major problem with sigmoid and tanh is:

- A) Non-differentiability
- B) Vanishing gradients
- C) High memory usage
- D) Sparse activation

✓ **Answer: B**

📝 **Explanation:** They saturate $\rightarrow$ gradients ≈ 0 .

9. Saturation in activation functions leads to:

- A) Large gradients
- B) Stable training
- C) Overconfident predictions
- D) Linear models

✓ **Answer: C**

📝 **Explanation:** Outputs approach 0 or 1 too strongly.

10. Sigmoid activation is mainly recommended for:

- A) Hidden layers
- B) Middle layers
- C) Output layers (probabilities)
- D) Convolutional layers

✓ **Answer: C**

📝 **Explanation:** Sigmoid outputs probabilities.

11. Softmax outputs:

- A) Binary values
- B) Independent probabilities
- C) A probability distribution
- D) Linear scores

✓ **Answer: C**

📝 **Explanation:** Softmax normalizes outputs to sum to 1.

12. The softmax function ensures:

- A) All outputs are zero
- B) $\sum$ outputs = 1
- C) Linear separability
- D) Convex optimization

✓ **Answer: B**

📝 **Explanation:** Softmax normalizes scores into probabilities.

13. Numerical stability in softmax is improved by:

- A) Removing exp
- B) Dividing by $\max(x)$
- C) Subtracting $\max(x)$
- D) Using sigmoid

 **Answer: C**

 **Explanation:** Subtracting max prevents overflow.

14. Default activation for hidden layers in modern networks:

- A) Sigmoid
- B) Tanh
- C) ReLU or GELU
- D) Softmax

 **Answer: C**

 **Explanation:** ReLU / GELU are modern defaults.

15. For regression tasks, the output layer should use:

- A) Sigmoid
- B) Softmax
- C) ReLU
- D) Linear activation

 **Answer: D**

 **Explanation:** Regression predicts continuous values.

16. For binary classification, the output layer uses:

- A) Softmax
- B) Linear
- C) Sigmoid
- D) Tanh

 **Answer: C**

 **Explanation:** Sigmoid outputs probability for binary tasks.

17. For multiclass classification, we use:

- A) One sigmoid neuron
- B) One linear neuron

- C) One neuron per class + softmax
- D) ReLU output

 **Answer:** C

 **Explanation:** Softmax over class scores.

18. Multilabel classification differs because it uses:

- A) Softmax
- B) Linear output
- C) Sigmoid per class
- D) One-hot labels only

 **Answer:** C

 **Explanation:** Classes are independent.

19. Any function can be a module if it is:

- A) Linear
- B) Convex
- C) Differentiable almost everywhere
- D) Probabilistic

 **Answer:** C

 **Explanation:** Gradients are required for learning.

20. Writing complex models as cascades of simple modules helps:

- A) Overfitting
- B) Debugging
- C) Slower training
- D) Removing gradients

 **Answer:** B

 **Explanation:** Easier to analyze and debug.

21. Neural network architecture refers to:

- A) Loss function
- B) Optimization method
- C) Overall structure and connections
- D) Activation function only

✓ **Answer:** C

📝 **Explanation:** Architecture = structure + connectivity.

22. Most neural networks are organized as:

- A) Fully connected graphs
- B) Cyclic graphs
- C) Chain of layers
- D) Trees

✓ **Answer:** C

📝 **Explanation:** Layers arranged sequentially.

23. If a deep network has no nonlinearities:

- A) It becomes unstable
- B) It is equivalent to one linear layer
- C) It cannot run
- D) It overfits

✓ **Answer:** B

📝 **Explanation:** Composition of linear layers is linear.

24. The correct quiz answer is:

“You could’ve just learned a single layer” because:

- A) Gradients vanish
- B) No loss function
- C) Linear layers collapse into one
- D) Overfitting occurs

✓ **Answer:** C

📝 **Explanation:** No nonlinearity → linear mapping.

25. Universal Approximation Theorem states that:

- A) Deep networks always generalize
- B) One hidden layer can represent any function
- C) Training always succeeds
- D) Depth is unnecessary

✓ **Answer:** B

📝 **Explanation:** With enough units, one hidden layer suffices.

26. Despite universal approximation:

- A) Training is guaranteed
- B) Overfitting is impossible
- C) Optimization may fail
- D) Depth is useless

 **Answer: C**

 **Explanation:** Learning parameters is still hard.

27. In worst case, shallow networks require:

- A) Linear number of units
- B) No hidden units
- C) Exponential number of units
- D) Fixed depth

 **Answer: C**

 **Explanation:** Some functions are inefficient for shallow nets.

28. Deeper networks often:

- A) Overfit more
- B) Require more units
- C) Generalize better
- D) Are always convex

 **Answer: C**

 **Explanation:** Hierarchical representations improve generalization.

29. Increasing parameters without increasing depth:

- A) Is always better
- B) Has little effect on performance
- C) Guarantees higher accuracy
- D) Removes overfitting

 **Answer: B**

 **Explanation:** Depth matters more than width alone.

30. Backpropagation is an application of:

- A) Probability chain rule
- B) Bayes rule
- C) Chain rule of calculus
- D) Linear algebra

✓ **Answer:** C

📝 **Explanation:** Backprop efficiently computes derivatives using chain rule.

🧠 MCQ – Computational Graphs & Backpropagation (30 Questions)

1. In a computational graph, each node represents:

- A) A weight
- B) A variable
- C) A gradient
- D) A layer

✓ **Answer:** B

📝 **Explanation:** Each node represents a variable or intermediate computation.

2. In a computational graph, edges represent:

- A) Variables
- B) Operations
- C) Inputs
- D) Outputs

✓ **Answer:** B

📝 **Explanation:** Edges correspond to operations applied to variables.

3. A simple operation in a computational graph can be:

- A) Addition
- B) Multiplication
- C) Matrix multiplication
- D) All of the above

✓ **Answer:** D

📝 **Explanation:** Any differentiable function can be an operation.

4. In backpropagation, gradients are computed using:

- A) Bayes theorem
- B) Chain rule of calculus
- C) Linear algebra only
- D) Eigen decomposition

✓ **Answer: B**

📄 **Explanation:** Backprop uses the chain rule to propagate derivatives.

5. For a composite function ($z = f(g(x))$), the chain rule gives:

- A) $(dz/dx = f(g(x)) + g(x))$
- B) $(dz/dx = df/dg * dg/dx)$
- C) $(dz/dx = f'(x) + g'(x))$
- D) $(dz/dx = df/dx * dg/dx)$

✓ **Answer: B**

📄 **Explanation:** $(dz/dx = dz/dy * dy/dx = f'(g(x)) * g'(x))$.

6. Backpropagation stores intermediate results to:

- A) Save memory
- B) Avoid recomputation
- C) Increase nonlinearity
- D) Prevent overfitting

✓ **Answer: B**

📄 **Explanation:** Intermediate values like $(f(w))$ are cached to compute gradients efficiently.

7. If a function is not differentiable at a point:

- A) Backprop cannot compute gradient exactly
- B) Gradient is infinite
- C) Chain rule still works
- D) Computation stops

✓ **Answer: A**

📄 **Explanation:** Differentiability is required for accurate gradient computation.

8. The derivative of $(y = x_1 * x_2)$ with respect to (x_1) is:

- A) (x_1)
- B) (x_2)
- C) $(x_1 + x_2)$
- D) 1

✓ **Answer:** B

📄 **Explanation:** $(\partial (x_1 * x_2) / \partial x_1 = x_2)$.

9. For $(L = y_1 + y_2)$, $(\partial L / \partial y_1 = ?)$

- A) (y_1)
- B) (y_2)
- C) 1
- D) 0

✓ **Answer:** C

📄 **Explanation:** Sum rule: derivative of y_1 is 1.

10. If $(M = \max(L, 0))$ and $(L < 0)$, then:

- A) $(dM/dL = 1)$
- B) $(dM/dL = 0)$
- C) $(dM/dL = -1)$
- D) Cannot be computed

✓ **Answer:** B

📄 **Explanation:** ReLU derivative is 0 when input < 0 .

11. Gradient of a scalar output w.r.t vector input is called:

- A) Jacobian
- B) Hessian
- C) Gradient vector
- D) Covariance

✓ **Answer:** C

📄 **Explanation:** For scalar output, gradient vector contains all partial derivatives.

12. Jacobian generalizes the gradient for:

- A) Vector inputs, scalar output
- B) Scalar inputs, vector output
- C) Vector inputs, vector output
- D) All of the above

✓ **Answer:** C

📄 **Explanation:** Jacobian ($J = dh/dx$) maps vector input to vector output.

13. Vectorizing a matrix (M) means:

- A) Flattening it into a vector
- B) Multiplying by identity
- C) Transposing only
- D) Computing determinant

 **Answer:** A

 **Explanation:** $\text{Vec}(M)$ stacks all elements into a single vector.

14. In Einstein notation, summation over repeated indices:

- A) Is optional
- B) Reduces loops in tensor computation
- C) Computes determinants
- D) Defines activation functions

 **Answer:** B

 **Explanation:** Einstein summation simplifies tensor operations.

15. Chain rule for tensors involves:

- A) Only scalar multiplication
- B) Summing all paths affecting output
- C) Ignoring dimensions
- D) Computing inverse matrices

 **Answer:** B

 **Explanation:** Each input element affects outputs via all paths.

16. Backpropagation efficiency depends on:

- A) Storing intermediate results
- B) Recomputing everything each step
- C) Ignoring gradients
- D) Using non-differentiable activations

 **Answer:** A

 **Explanation:** Cache intermediate computations to avoid redundant work.

17. Gradient of sum ($f(x) + g(x)$) is:

- A) $(f'(x) * g'(x))$
- B) $(f'(x) + g'(x))$

C) $(f'(x) - g'(x))$

D) Zero

✓ **Answer:** B

📝 **Explanation:** Sum rule: derivatives add.

18. Gradient of product $(f(x) * g(x))$ is:

A) $(f'(x) * g'(x))$

B) $(f(x) * g'(x) + g(x) * f'(x))$

C) $(f(x) + g(x))$

D) $(f'(x) / g'(x))$

✓ **Answer:** B

📝 **Explanation:** Product rule: $(d(fg)/dx = f * g' + g * f')$.

19. Partial derivatives depend on:

A) One path only

B) All paths from variable to output

C) Random noise

D) Activation type only

✓ **Answer:** B

📝 **Explanation:** Must sum contributions from all paths in graph.

20. Computational graph for $(z = x * y + w)$ requires:

A) Nodes for x, y, w only

B) Nodes for x, y, w, and intermediate xy

C) Nodes for output only

D) No nodes needed

✓ **Answer:** B

📝 **Explanation:** Intermediates like xy need nodes for backprop.

21. Storing repeated subexpressions:

A) Saves memory

B) Reduces computation

C) Speeds up gradient computation

D) All of the above

✓ **Answer:** D

📝 **Explanation:** Avoids recomputation and improves efficiency.

22. Differentiability is important because:

- A) Without it, network cannot learn
- B) It increases overfitting
- C) It improves regularization
- D) Reduces forward computation

✓ **Answer:** A

📝 **Explanation:** Gradients are undefined for non-differentiable points.

23. ReLU is non-differentiable at:

- A) $x > 0$
- B) $x < 0$
- C) $x = 0$
- D) All x

✓ **Answer:** C

📝 **Explanation:** ReLU derivative is undefined at zero.

24. Jacobian determinant measures:

- A) Rate of change in space
- B) Loss function value
- C) Learning rate
- D) Gradient norm

✓ **Answer:** A

📝 **Explanation:** Measures local volume change under transformation.

25. For a row vector input ($x \in \mathbb{R}^{1 \times n}$), gradient dimension:

- A) Scalar
- B) Same as x
- C) Twice the size of x
- D) n^2

✓ **Answer:** B

📝 **Explanation:** Gradient has same shape as input vector.

26. Chain rule for multiple operations in graph:

- A) Multiply all Jacobians along paths
- B) Add all Jacobians
- C) Ignore intermediate variables
- D) Only consider first operation

 **Answer: A**

 **Explanation:** Chain rule multiplies derivatives along computation paths.

27. Backprop on high-dimensional tensors requires:

- A) Scalar derivatives only
- B) Tensor multiplication
- C) Ignoring shapes
- D) Only diagonal entries

 **Answer: B**

 **Explanation:** Must account for all dimensions in Jacobians.

28. Gradient vectors are:

- A) Always scalars
- B) Tensors containing all partial derivatives
- C) Determinants
- D) Probability distributions

 **Answer: B**

 **Explanation:** Encodes sensitivity of output to every input dimension.

29. Vectorizing gradients helps by:

- A) Simplifying storage and computation
- B) Removing dependencies
- C) Increasing nonlinearity
- D) Preventing saturation

 **Answer: A**

 **Explanation:** Flatten matrices to work with standard vector operations.

30. In backpropagation, each partial derivative contributes:

- A) To a single output only
- B) To all paths from input to output

- C) To a single input only
- D) Randomly

✓ **Answer:** B

📝 **Explanation:** Every path from input to output contributes to derivative.

🧠 MCQ – Computational Graphs & Backprop (30 Questions)

1. In a forward computational graph, we:

- A) Compute gradients
- B) Compute activations of each module
- C) Store only outputs
- D) Ignore intermediate variables

✓ **Answer:** B

📝 **Explanation:** Forward pass computes activations ($h = h_w(x)$) for each module.

2. After computing the activation in a module, we usually:

- A) Discard it
- B) Store it for backprop
- C) Only use it for output
- D) Multiply by learning rate

✓ **Answer:** B

📝 **Explanation:** Intermediate activations are needed for backpropagation.

3. Forward pass is repeated:

- A) Only once for final output
- B) Recursively and in the correct order
- C) Randomly
- D) Only for first layer

✓ **Answer:** B

📝 **Explanation:** Forward pass must propagate through all modules in order.

4. Setting $x := h$ in a forward graph means:

- A) Updating the module weights
- B) Using current module output as next input

- C) Computing gradient
- D) Ignoring input

 **Answer: B**

 **Explanation:** Output of one module becomes input for the next.

5. Storing intermediate variables in forward pass:

- A) Saves memory
- B) Saves computation time
- C) Prevents overfitting
- D) Reduces learning rate

 **Answer: B**

 **Explanation:** Avoids recomputation during backward pass; costs memory.

6. Trade-off in storing activations:

- A) More memory, less recomputation
- B) Less memory, more recomputation
- C) Both A & B
- D) None

 **Answer: C**

 **Explanation:** Forward activations save computation but consume memory.

7. Backward pass uses:

- A) Forward activations and gradient functions
- B) Only input data
- C) Only final loss
- D) Random numbers

 **Answer: A**

 **Explanation:** Reverse-mode differentiation requires stored activations.

8. Reverse-mode automatic differentiation is called so because:

- A) Gradients flow in reverse of data
- B) Data flows backwards
- C) Activations are ignored
- D) Learning rate is reversed

✓ **Answer:** A

📝 **Explanation:** Backprop flows gradients backward along the computational graph.

9. During backprop, gradients are summed over:

- A) All modules
- B) All samples in mini-batch
- C) Only last layer
- D) Only first layer

✓ **Answer:** B

📝 **Explanation:** Gradients from all mini-batch samples are accumulated.

10. To implement backprop, modules must have:

- A) Only forward function
- B) Gradient function w.r.t inputs and weights
- C) Random initialization
- D) No derivative

✓ **Answer:** B

📝 **Explanation:** Gradient functions are essential for computing local derivatives.

11. Forward computational graph stores:

- A) Only final output
- B) All intermediate activations
- C) Gradients
- D) Only inputs

✓ **Answer:** B

📝 **Explanation:** Needed to efficiently compute backward gradients.

12. Forward pass order is:

- A) Topologically correct order of modules
- B) Random
- C) Reverse of backward
- D) Only input layer first

✓ **Answer:** A

📝 **Explanation:** Correct module order ensures valid computations.

13. In backprop, each module computes gradient w.r.t:

- A) Its input and parameters
- B) Only input
- C) Only output
- D) Nothing

 **Answer: A**

 **Explanation:** Local gradients are computed for both weights and inputs.

14. Computational graph edges represent:

- A) Data flow
- B) Gradient flow only
- C) Activation functions
- D) Loss function

 **Answer: A**

 **Explanation:** Edges show dependencies of module outputs on inputs.

15. Forward pass can be considered:

- A) Data flow graph
- B) Gradient graph
- C) Random graph
- D) Loss function

 **Answer: A**

 **Explanation:** Forward pass computes activations along graph edges.

16. Reverse graph is used to:

- A) Compute gradients efficiently
- B) Compute outputs
- C) Initialize weights
- D) Reduce mini-batch size

 **Answer: A**

 **Explanation:** Backward pass follows reverse graph to propagate gradients.

17. Sum of gradients in mini-batch ensures:

- A) Correct gradient estimation
- B) Faster convergence

- C) Smaller memory
- D) No need for learning rate

✓ **Answer:** A

📝 **Explanation:** Accumulates contribution from all samples.

18. Forward activations are needed for:

- A) ReLU derivatives
- B) Sigmoid derivatives
- C) All differentiable functions
- D) None

✓ **Answer:** C

📝 **Explanation:** Gradients often depend on intermediate activations.

19. Reverse-mode AD is most efficient when:

- A) Output is scalar
- B) Input is scalar
- C) Both input/output large
- D) Input/output are zero

✓ **Answer:** A

📝 **Explanation:** Gradient w.r.t many inputs to single scalar output is efficient.

20. Without storing forward activations:

- A) Backprop requires recomputation
- B) Backprop is impossible
- C) Forward pass fails
- D) Mini-batch cannot be used

✓ **Answer:** A

📝 **Explanation:** Recomputation trades off memory for compute time.

21. In forward graph, recursive computation:

- A) Uses previous outputs as next inputs
- B) Uses gradients immediately
- C) Skips first layer
- D) Only works for scalar outputs

✓ **Answer:** A

📝 **Explanation:** Forward activations propagate recursively through the network.

22. Each node in computational graph represents:

- A) Module output or variable
- B) Weight only
- C) Input only
- D) Loss only

✓ **Answer:** A

📝 **Explanation:** Nodes correspond to activations or variables.

23. Each edge in computational graph represents:

- A) Dependency of one variable on another
- B) Weight
- C) Mini-batch index
- D) Learning rate

✓ **Answer:** A

📝 **Explanation:** Shows how outputs depend on inputs or previous activations.

24. Backprop efficiency relies on:

- A) Reusing intermediate activations
- B) Computing full Jacobians
- C) Ignoring chain rule
- D) Using only first layer

✓ **Answer:** A

📝 **Explanation:** Saves computation and memory.

25. Gradient w.r.t weights in a module:

- A) Local computation using input and upstream gradient
- B) Global computation using full Jacobian
- C) Only forward pass
- D) None

✓ **Answer:** A

📝 **Explanation:** Each module computes local gradient using chain rule.

26. Reverse-mode AD is also called:

- A) Backpropagation
- B) Forward propagation
- C) Stochastic gradient descent
- D) Linear regression

 **Answer: A**

 **Explanation:** Standard backprop uses reverse-mode automatic differentiation.

27. Computational graph abstraction allows:

- A) Modular gradient computation
- B) Efficient memory usage
- C) AutoDiff frameworks to work
- D) All of the above

 **Answer: D**

 **Explanation:** Graphs make backprop scalable, modular, and efficient.

28. Storing forward activations costs:

- A) Memory
- B) Time
- C) Accuracy
- D) Learning rate

 **Answer: A**

 **Explanation:** Storing activations consumes RAM, but saves recomputation.

29. Forward pass + reverse pass gives:

- A) Network outputs and gradients
- B) Only network outputs
- C) Only weights
- D) Learning rate

 **Answer: A**

 **Explanation:** Full backprop computes outputs and gradients for optimization.

30. Key idea of computational graph:

- A) Modular forward + local backward + efficient gradient flow
- B) Ignore chain rule

- C) Only scalar outputs
- D) Random weight updates

✓ **Answer:** A

📝 **Explanation:** Graph abstraction allows structured, memory-efficient backprop.

🧠 MCQ – Reverse Graph & Backpropagation (30 Questions)

1. Reverse computational graph is used to:

- A) Compute activations
- B) Compute gradients efficiently
- C) Initialize weights
- D) Store inputs

✓ **Answer:** B

📝 **Explanation:** Reverse graph allows computing gradients via backpropagation efficiently.

2. Backpropagation uses:

- A) Forward activations
- B) Only weights
- C) Random numbers
- D) Output labels only

✓ **Answer:** A

📝 **Explanation:** Gradients often depend on activations from forward pass.

3. Reverse-mode automatic differentiation flows:

- A) Forward along data
- B) Reverse along data flow
- C) Randomly
- D) Only through input layer

✓ **Answer:** B

📝 **Explanation:** Gradients propagate backward, opposite to forward data flow.

4. Gradient functions must be implemented w.r.t:

- A) Inputs and weights
- B) Inputs only

- C) Weights only
- D) Outputs only

✅ **Answer:** A

📝 **Explanation:** Needed for each module to compute local gradients.

5. Gradients in mini-batch are computed by:

- A) Summing over all samples
- B) Taking last sample only
- C) Multiplying outputs
- D) Ignoring batch

✅ **Answer:** A

📝 **Explanation:** Sum ensures proper gradient for all samples in batch.

6. Backpropagation is essentially:

- A) Chain rule applied through network
- B) Matrix multiplication only
- C) Random initialization
- D) Forward pass only

✅ **Answer:** A

📝 **Explanation:** Each gradient is a product of local derivatives along the path.

7. Storing forward activations helps:

- A) Avoid recomputation during backward pass
- B) Increase learning rate
- C) Reduce mini-batch size
- D) Skip chain rule

✅ **Answer:** A

📝 **Explanation:** Caching saves computation time at the cost of memory.

8. The backward gradient of the loss w.r.t weight (w) is:

- A) $(dL/dw = dL/dh \cdot dh/dw)$
- B) $(dL/dw = h)$ only
- C) $(dL/dw = L)$ only
- D) $(dL/dw = w)$ only

✓ **Answer:** A

📝 **Explanation:** Chain rule gives gradient w.r.t each parameter.

9. Local gradient for sigmoid σ is:

- A) $\sigma(1 - \sigma)$
- B) σ^2
- C) $1 - \sigma$
- D) σ

✓ **Answer:** A

📝 **Explanation:** Sigmoid derivative: $(d\sigma/dx = \sigma(x) \cdot (1 - \sigma(x)))$.

10. Gradient w.r.t hidden activation h in last layer is:

- A) $(dL/dh = -(y^{**} - h))$
- B) $(dL/dh = y^{**} + h)$
- C) $(dL/dh = h^2)$
- D) $(dL/dh = 0)$

✓ **Answer:** A

📝 **Explanation:** For MSE loss, gradient w.r.t output is difference between target and prediction.

11. Backpropagation efficiency relies on:

- A) Caching forward activations
- B) Ignoring chain rule
- C) Random initialization
- D) Only using first layer

✓ **Answer:** A

📝 **Explanation:** Avoids redundant computation across layers.

12. Each module in reverse graph computes gradient w.r.t:

- A) Its input and weights
- B) Only input
- C) Only output
- D) None

✓ **Answer:** A

📝 **Explanation:** Local computation using chain rule.

13. Reverse-mode AD is efficient for:

- A) Many inputs → single scalar output
- B) Single input → many outputs
- C) Single input → single output
- D) Random inputs

 **Answer:** A

 **Explanation:** Typical neural networks have many parameters (inputs) and single loss (scalar output).

14. Backpropagation step 1:

- A) Compute forward activations recursively
- B) Compute gradients
- C) Initialize weights
- D) Skip layers

 **Answer:** A

 **Explanation:** Forward pass needed to store activations before backward pass.

15. Backpropagation step 2:

- A) Compute gradients along reverse path
- B) Forward compute outputs
- C) Random weight updates
- D) Skip activations

 **Answer:** A

 **Explanation:** Start from last layer and propagate gradients backward.

16. Backpropagation step 3:

- A) Update weights using gradients
- B) Forward propagate again
- C) Store inputs only
- D) Compute loss only

 **Answer:** A

 **Explanation:** Use computed gradients in SGD or optimizer to update parameters.

17. Gradient of loss w.r.t weight in hidden layer uses:

- A) Chain rule: $dL/dh \cdot dh/dw$
- B) Only input value

- C) Only output value
- D) Learning rate

✅ **Answer:** A

📝 **Explanation:** Each layer multiplies upstream gradient by local derivative.

18. Why store activations in forward pass?

- A) Needed for derivatives
- B) Reduce learning rate
- C) Random initialization
- D) Save loss only

✅ **Answer:** A

📝 **Explanation:** Gradients often depend on activations of each layer.

19. Backpropagation is “simple but complicated” because:

- A) Chain rule is simple, efficient implementation tricky
- B) It requires no math
- C) Only uses random numbers
- D) Learning rate is zero

✅ **Answer:** A

📝 **Explanation:** Conceptually chain rule, but efficient code and memory management is complex.

20. Forward pass formula for one layer with sigmoid:

- A) $h = \sigma(wh)$
- B) $h = w + h$
- C) $h = h^2$
- D) $h = L$

✅ **Answer:** A

📝 **Explanation:** Forward propagation computes activation via weights and activation function.

21. Loss function example in backprop slides:

- A) $L = 0.5 \cdot (l - h)^2$
- B) $L = h^2$
- C) $L = w^2$
- D) $L = \text{random}$

✓ **Answer:** A

📝 **Explanation:** Standard MSE loss used to illustrate backprop.

22. Upstream gradient is:

- A) Gradient received from next layer
- B) Input itself
- C) Weight only
- D) Random number

✓ **Answer:** A

📝 **Explanation:** Each layer multiplies upstream gradient by local derivative.

23. Gradient for hidden layer:

- A) $dL/dh = dL/dh_{next} \cdot dh_{next}/dh$
- B) $dL/dh = h^2$
- C) $dL/dh = L$ only
- D) $dL/dh = 0$

✓ **Answer:** A

📝 **Explanation:** Chain rule propagates gradient through network.

24. Stochastic Gradient Descent uses:

- A) dL/dw to update weights
- B) Forward activations to update weights
- C) Only loss to update weights
- D) Random numbers

✓ **Answer:** A

📝 **Explanation:** Weights updated along negative gradient of loss.

25. Backprop visualization shows:

- A) Forward activations and backward gradients
- B) Only inputs
- C) Only weights
- D) Only loss

✓ **Answer:** A

📝 **Explanation:** Diagram shows how gradients flow opposite to data flow.

26. Local derivative for sigmoid layer in backprop:

- A) $h \cdot (1 - h)$
- B) σ
- C) $1 - \sigma$
- D) 0

 **Answer:** A

 **Explanation:** Standard derivative for chain rule computation.

27. Gradient for MSE loss w.r.t output:

- A) $-(y^* - h)$
- B) h^2
- C) y^*
- D) 0

 **Answer:** A

 **Explanation:** Direct derivative of $0.5 (y - h)^2$ w.r.t h .

28. Efficient backprop requires:

- A) Caching activations, avoiding redundant operations
- B) Ignoring chain rule
- C) Random initialization
- D) Only last layer

 **Answer:** A

 **Explanation:** Saves compute time and allows scalable training.

29. Main idea of reverse graph:

- A) Local gradients + chain rule + backward propagation
- B) Forward propagation only
- C) Random updates
- D) Skip layers

 **Answer:** A

 **Explanation:** Each module computes gradient w.r.t its input and weights.

30. Backprop works theoretically because:

- A) Chain rule applies even in non-convex networks
- B) Networks are always linear

C) Gradients are random

D) Loss is zero

✓ **Answer: A**

📖 **Explanation:** Chain rule is valid for any differentiable composition of functions.

ملخص

1 Linear vs Non-linear

- $\text{ReLU} = \max(0, x)$ الفيلكتور بتاع المدخلات، الـ x ماتريكس و A ، مع $f = \text{ReLU}(Ax)$ زي **Linear** لو عندنا دالة
- XOR لوحدها محدودة: مش هتقدر تمثل حاجات معقدة زي Linear models
- (shallow يعني عميقة بدل) **non-linear** كده نقدر نعمل دوال $\rightarrow \text{ReLU} + \text{الحل}$: نحط طبقات أكثر

2 Feedforward Neural Networks (MLPs)

- اسمها كده عشان المعلومات بتمشي لقدام بس، من المدخلات للمخرجات
- زي الـ RNNs الـ feedback مفيش
- input صح لكل output تقرب (f^{**}) function هدفها: تتعلم

شكلها:

$x \rightarrow \text{hidden layer 1} \rightarrow \text{hidden layer 2} \rightarrow \dots \rightarrow \text{output } y$

- ReLU زي **Activation function** وبتطبق **Bias** و **Weights** كل طبقة ليها

3 Formula / Notation

- **composite function:** الشبكة عبارة عن
[
 $y = f(x; \theta) = h^{\{L\}} \circ h^{\{L-1\}} \circ \dots \circ h^{\{1\}}(x)$
]
- $(\theta^{\{l\}})$ عنده معلومات $(h^{\{l\}})$ كل
- XOR مثال لحل:
[
 $f(x; W, c, w, b) = w^T \max(0, Wx + c) + b$
]

4 Modules (Building Blocks)

- module بنسبها transformation كل طبقة أو
 - activation function باستخدام output ويرجع input بياخذ Module
 - أو لا trainable parameters ممكن يكون عنده
-

5 Non-linear Feature Learning

- محدودة → Logistic Regression بسيط زي Linear model لو عندنا
 - feature transformation $\phi(x)$ العمل: نعمل
 - بدل ما نحددنا $\phi(x)$ بتعلم Deep learning
 - linearly separable features الهدف: نخلي الـ
 - learned feature space ممكن يتحل بخط مستقيم في الـ neural network الأصلي، بس بعد input ما ينفش بالـ XOR: مثال
-

6 Requirements

- Activation functions تقريباً دائماً 1st-order differentiable لازم تكون
 - ناخذ بالنا → architecture في cycles أو loops لو في
-

7 Deep Learning خلاصة استراتيجية

- family of functions $\phi(x; \theta)$ واحدة صح، بنصم function بدل ما نركز على
 - كويس Features الشبكة بتتعلم تمثيل
 - دي features عادي يقدر يشتغل على الـ linear model بعد كده
-

تمام! خلينا نلخصك كل النقاط اللي بعتهالك بطريقة سهلة وبالعامة

1 Mixing hierarchies / DAG & skip connections

- ساعات الشبكة بتبقى معقدة وبدل ما نعمل طبقة ورا طبقة بس، ممكن نخلط الطبقات ببعض
 - (LIDAR + بيانات RGB مثلاً صور) ده مفيد لو عارفين كويس مشكلة الداتا أو لو عايزين ندمج مصادر مختلفة
 - (نتخطى طبقات) أو نخلط skip connections ممكن يبقى فيه chain مش مجرد Directed Acyclic Graph الرسم بيبقى الداتا من أكثر من مكان
-

2 Data-efficient modules & hierarchies

- مقابل عدد التدريب module في شبكة كبيرة لازم نفكر كويس في تعقيد كل.
 - أحيانًا موديل بسيط لكن بتدربه كثير يدي نتائج أحسن من موديل كبير نصه تدريب.
 - SoTA. لكنه سريع في التدريب وعشان كده (half-linear) بسيط ReLU.
 - للطبقة اللي بعدها output بالترتيب الصح عشان يقدر يمرر module لازم نستخدم الذاكرة والمعلومات بذكاء، ونحسب كل.
-

3 Loopy connections / Recurrent

- loop للمستقبل → فيه input يتاعه ك output لو موديل بياخد.
 - RNN. عشان التدريب يمشي → ده أساس ("unfold") loop لازم نفك الـ.
 - feedforward. كثير في Loopy connections دلوقتي عادةً مش بنستخدم.
-

4 Cost functions

- مباشر linear مفيش حل → non-convex تبقى loss function non-linear الشبكات العميقة.
 - SGD زي gradient-based optimization بنستخدم.
 - output discrete لو cross-entropy وده تقريبًا → negative log-likelihood بنوصل لـ → training set على Max likelihood.
 - Mean Squared Error (MSE) بنرجع للـ → Gaussian model output مثال: لو.
 - كبير وواضح عشان التعلم يمشي كويس gradient الهدف: يكون.
 - بيحل المشكلة negative log-likelihood → صغير → صعب التدريب gradient بتخلي (flat تبقى) saturate اللي Functions. أحيانًا.
-

5 Main point عن cost functions

- هي اللي بتحدد ايه اللي الموديل المفروض يعمل Cost function.
 - يعرف يمشي في اتجاه صح optimizer عشان → يكون كبير ومتوقع gradient لازم.
 - يصغر → صعب التدريب gradient → activation saturate لو.
 - Negative log-likelihood (softmax output زي) exponentiation بيحل مشكلة.
 - Cross-entropy Logistic regression بعض النماذج زي → minimum مش دايماً ليها توصله.
-

6 Cost functions أنواع

- Regression → **Euclidean loss / MSE** → للـ (كبيرة تتضرب تربيعي errors) حساس للـ.
 - Classification → **Cross-entropy, KL-divergence**.
-

7 Activation functions

- **وظيفتها** node من كل weighted sum تحول: **وظيفتها** → output.
 - **squashing function** محدود → نسميها output لو.
 - اختيارها مهم جدًا على الأداء والقدرة على تعلم الشبكة.
 - **activation** تستخدم نفس hidden layers معظم.
 - أغلب الوقت **differentiable** لازم تكون.
-

8 Linear Units

- مستقر learning → ثابت gradient → saturation لا يوجد.
 - Formula: $h(x; w) = x \cdot w + b$
 - Gradient لـ x : بالنسبة لـ $(dh/dx = w)$
-

9 ReLU

- $h(x) = \max(0, x)$
- Gradient = 1 لو $x > 0$, 0 لو $x \leq 0$
- **Advantages:**
 - Sparse activation → 50% units active randomly
 - Better gradient propagation → أقل vanishing gradient
 - Efficient computation → بس مقارنة + جمع + ضرب
 - Scale-invariant
- **Problems:**
 - عند 0 (بس نقدر نختار 0 أو 1) Non-differentiable
 - Not zero-centered
 - Unbounded
 - Dead neurons → بعض الوحدات ممكن تبطل تشتغل

- **ReLU** default non-linearity هو دلو قتي

1 0 Leaky ReLU

- dead neurons: عشان يحل مشكلة
[
$$h(x) = \begin{cases} x & x > 0 \\ ax & x \leq 0 \end{cases}$$

]
• active مش unit صغير بس إيجابي لو gradient $\rightarrow$ صغير a
• Parametric ReLU (PReLU) $\rightarrow a$ بيتعلم كمعامل

1 1 ELU (Exponential Linear Unit)

- Smooth approximation $\downarrow$ ReLU
[
$$h(x) = \begin{cases} x & x > 0 \\ \exp(x) - 1 & x \leq 0 \end{cases}$$

]
• $x < 0$ لو non-monotonic bump عندها
• BERT مستخدمة في موديلات زي

تمام 🙌 هشرحك كل النقط دي بشكل مختصر وبالعامية ومنظم عشان تثبت

1 GELU (Gaussian Error Linear Unit)

- ELU و ReLU زي **Activation Function** دي
- ممكن يغير اتجاهه (الـ gradient) يعني الميل **monotonic** بس مش ELU شبه
- بتدي أداء أحسن في الموديلات الحديثة
- **Vision Transformers** في default هي الـ
- state-of-the-art models مستخدمة في

👉 الخلاصة:

في موديلات كبيرة ReLU اختيار حديث وقوي بدل = GELU

2 Sigmoid و Tanh

◆ Tanh

- لـ $1+1$ ناتجها من
- الداتا بتطلع متوسطة حوالين صفر
- ده يساعد إن الجريديننت يبقى أقوى
- أقل للطبقات اللي بعدها bias

◆ Sigmoid

- لـ 0 ناتجها من
- مناسبة تمثل احتمالات

✗ مشاكلهم

- الانتين بيتشبعوا عند القيم الكبيرة
- الجريديننت يقرب من صفر
- **vanishing gradient** يحصل
- hidden layers مش كويسين للـ

👉 الخلاصة:

- في الطبقات الوسطى Sigmoid أحسن من Tanh
- output layer أحسن في الـ Sigmoid

3 المعادلات (ببساطة)

- **Sigmoid:**
بتحول أي رقم لاحتفال بين 0 و 1
- **Tanh:**
بس بين -1 و $1+1$ Sigmoid نفس فكرة أصلاً Sigmoid وبتطلع من

4 Softmax

- بتحول الأرقام لاحتفالات
- مجموعهم دائماً = 1
- **Multi-class classification** تستخدم في

ليه بنطرح أكبر قيمة؟

- عشان الاستقرار العددي
- exp نتجنب أرقام كبيرة جداً في

👉 الخلاصة:

Softmax = توزيع احتمالات على أكثر من كلاس

5 Activation Function اختيار

◆ Hidden Layers

- ReLU أو GELU: الأفضل
- Sigmoid أو Tanh في RNNs

◆ Output Layer

- Regression → Linear
- Binary Classification → Sigmoid
- Multiclass → Softmax
- Multilabel → Sigmoid

⚠ ملحوظة مهمة:

inference مش دائماً نفس الحاجة في التدريب والد

6 Modules (الوحدات)

- Module أي فانكشن قابلة للاشتقاق تقريباً ينفع تبقى
- Module جو Module تقدر تركيب

مثال:

- ReLU → Tanh
 - أحسن تكتبهم خطوات ورا بعض
 - debugging أسهل في الفهم والد
-

7 Architecture Design (تصميم الشبكة)

- يعني:
 - عدد الوحدات
 - متوصلة إزاي
- (Chain) الشبكة بتبقى طبقات ورا بعض

مثال:

- Layer 1:
 $h(1) = g(Wx + b)$
 - Layer 2:
 $h(2) = g(W h(1) + b)$
-

8 سؤال الكويز (مهم جدًا)

non-linearities: لو عملت شبكة عميقة من غير

✓ الصبح:

واحدة بس Layer كأنك عامل

ليه؟

- واحد non-linearity = Linear Transformation من غير matrices لأن ضرب
-

9 Width vs Depth

◆ Universal Approximation

- شبكة بطبقة واحدة ممكن تمثل أي فانكشن
- بس محتاجة وحدات كتير جدًا

◆ المشكلة

- التدريب ممكن يفشل
 - Overfitting
 - الحل مش دائمًا عملي
-

10 ليه العمق مهم؟

- الشبكات العميقة:
 - وحدات أقل
 - تعميم أحسن
 - أخطاء أقل
 - كل طبقة تتعلم مستوى:
 - بسيطة features: طبقات أولى
 - معقدة patterns: طبقات أعمق
-

1 1 العمق أهم من عدد البراميتر

- تزود البراميتر بس = مش كفاية
- العمق الحقيقي هو اللي يفرق
- overfitting الشبكات العميقة بتستحمل براميتر أكثر من غير

1 2 أنواع الشبكات (Neural Network Jungle)

- MLP
- CNN
- RNN / LSTM / GRU
- Autoencoders
- GANs
- Transformers

👉 أهمهم عمليًا:

MLP – CNN – VAE – Transformers – LSTM

1 3 Chain Rule (قاعدة السلسلة)

- بتستخدم لحساب المشتقات
- لو فانكشن جوه فانكشن

مثال:

- $x \rightarrow g(x) \rightarrow f(g(x))$

المشتقة:

$$dz/dx = dz/dy \times dy/dx$$

👉 Chain Rule كله مبني على Backpropagation

🔥 الخلاصة النهائية:

- مهمة جدًا Activation Functions
- العمق أهم من العرض
- أساسية non-linearities
- Backprop = Chain Rule

حاضر 📖

خطوة Computational Graph → Gradients → Jacobian → Backpropagation ده شرح مبسط وبالعامية لكل الجزء ده من
خطوة، كأنك بتذاكري قبل الامتحان 📌

1 Computational Graph يعني إيه

- هو رسم بيمثل الحسابات اللي بتحصل في الشبكة
- كل Node = متغير (x, y, w, b ...)
- كل Edge / عملية = عملية حسابية (ReLU، Sigmoid...)، ضرب،

ليه بنستخدمه؟ 📌

- عشان:
 - نفهم الحسابات
 - بسهولة gradients نحسب الـ
 - نطبق Backpropagation

2 Computational Graph العمليات في الـ

- العمليات بسيطة:
 - +
 - ×
 - dot
 - matmul
 - ReLU
 - Sigmoid

مثال:

$$z = x * y$$

$$\hat{y} = \sigma(z)$$

يتحول لرسم فيه:

- $x \rightarrow$
- $y \rightarrow (x) \rightarrow z \rightarrow \sigma \rightarrow \hat{y}$

3 ليه بندي أسماء لقيم وسيطة؟

- في المعادلات مش كل خطوة ليها اسم
- **Node** لازم كل حاجة تبقى graph لكن في الـ
- فينسميها:
 - $u(1), u(2), u(3)...$

يعرف يرجع خطوة خطوة Backprop ده مهم عشان 

4 Chain Rule في Graph مثال بسيط على

لو:

$$y1 = x1 * x2$$

$$y2 = x1 - 3$$

$$L = y1 + y2$$

نحسب:

$$dL/dx1$$

بالـ Chain Rule:

$$dL/dx1 = dL/dy1 * dy1/dx1 + dL/dy2 * dy2/dx1$$

- $dL/dy1 = 1$
- $dy1/dx1 = x2$
- $dL/dy2 = 1$
- $dy2/dx1 = 1$

النتيجة: 

$$dL/dx1 = x2 + 1$$

5 ليه العمليات لازم تكون قابلة للاشتقاق؟

مثال Rosenblatt (>) :

$$M = (L > 0)$$

- (صح / غلط) **decision** دي عملية
- مشتقتها = 0

النتيجة: 

- gradient مفيش
- مفيش تعلم

📌 عشان كده:

Differentiability = التعلم

6 يعمل إيه؟ Backpropagation

- بيحسب:
 - dL/dw
 - dL/db
- وده عن طريق:
 - Chain Rule
 - من آخر الشبكة للأول

📌 Chain Rule تطبيق ذكي وسريع للـ Backprop

7 ذكي؟ Backprop ليه

لأنه:

- مرة واحدة intermediate value بيحسب كل
- ويخزنها
- بدل ما يعيد الحساب كذا مرة

👉 أسرع + أوفر ميموري

8 يعني إيه؟ Gradient

- لو عندك:

$h(x_1, x_2, \dots, x_n)$

الـ Gradient:

$\nabla_x h = [\partial h / \partial x_1, \partial h / \partial x_2, \dots]$

📌 يعني:

- على الناتج input تأثير كل
-

9 يعني إيه؟ Jacobian

- Gradient تعميم للـ
- لما:
 - Vector المدخلات
 - Vector والمخرجات

الحالات:

نستخدم Input Output

Scalar Scalar Derivative

Vector Scalar Gradient

Scalar Vector Jacobian

Vector Vector Jacobian

📌 Jacobian = Matrix مشتقات

10 بشكل هندسي Jacobian

- ييوصف:
 - إزاي الفضاء بيتشوه
 - غير خطي transformation تحت

زي:

- derivative = ميل الخط
 - Jacobian = تشويه المساحة
-

1 1 Vectorization

- Tensor لما نشغل بماتريكس أو
- Vector بنحولهم لـ
- عشان نحسب المشتقات

مثال:

$\text{Vec}(M) = [m_{11}, m_{12}, m_{13}, \dots]$

1 2 قواعد الاشتقاق المهمة

◆ Sum Rule

$$d(f + g)/dx = df/dx + dg/dx$$

◆ Product Rule

$$d(fg)/dx = f g' + g f'$$

1 3 Chain Rule في الشبكات

لو:

$$h = h_3(h_2(h_1(x)))$$

الاشتقاق:

$$dh/dx = dh_3/dh_2 \cdot dh_2/dh_1 \cdot dh_1/dx$$

📌 شبه ضرب المصفوفات

1 4 Chain Rule مع Tensors

- نفس الفكرة
- بس:
 - بنجمع على كل المسارات
 - indices وكل الـ

📌 يتسهّل ده جدًا Einstein notation

1 5 Backprop المهم في

- partial derivative: كل
 - input للـ output يعتمد على كل المسارات من
 - Backprop:
 - يجمع تأثير كل المسارات
-

1 6 Backpropagation – الخلاصة

- Forward pass:
 - نحسب $\hat{y}$

- Backward pass:
 - نحسب gradients
- Update:
 - نعدّل weights

🚩 من غير Backprop:
الشبكة لا تتعلم

🧠 ملخص في سطرين:

- Computational Graph = تمثيل الحسابات
- Backprop = Chain Rule + Graph
- Differentiability = التعلم
- Jacobian = Gradient متقدم

👉 تمام، خلّيني أكمل الشرح خطوة خطوة وبالعامة وربطه بكل السلايز اللي بعثتها، عشان الفكرة تثبت مش حفظ بس

🧠 Backpropagation = Chain Rule (بس بطريقة ذكية)

💡 الفكرة الأساسية

- (Composite functions) عبارة عن دوال مركبة Neural Network الـ
 - يعني:

$$\mathcal{L} = f_L(f_{L-1}(\dots f_1(x)))$$
 - عشان ندرّب الشبكة، محتاجين:

$$\frac{d\mathcal{L}}{dw_l}$$
- loss معينة على الـ layer تأثير وزن في: يعني

💡 Backprop إيه بالضبط؟

دايمًا بيرجع لنفس الصيغة

[
 $\frac{d\mathcal{L}}{dw}$

$$\frac{d\mathcal{L}}{dw} \cdot \frac{dh}{dw}$$

وودي مهمة جدًا

- جاي من اللي بعدك **Gradient**: الجزء الأول
- module بتاعك إنت كـ **محلي (Local) Gradient**: الجزء الثاني

"بيشوف حاله" Layer كل

Very local rule

◆ **Rule؟ مش بس Algorithm = Backprop** ليه بنقول

لأن:

- إحنا مش بنحسب مشتقات من الصفر كل مرة
- بنعيد استخدام حسابات قديمة (recursive reuse)

يعني:

$$\left[\frac{d\mathcal{L}}{dh_l} \cdot \frac{d\mathcal{L}}{dh_{l+1}} \right]$$

🔗 Output layer → layer قبلها → ... → input

◆ **Forward vs Backward**

- Forward pass:** loss والـ output نحسب الـ
- Backward pass:** Chain Rule باستخدام gradients نحسب الـ

❓ **أصلاً؟ Backprop** ليه نستخدّم

👉 Quiz السلايد حاطط

نحلّه سوا

1 هو أدق طريقة للتدريب؟

❌ لا، في طرق ثانية بس أقل كفاءة

2 هل المخ يتعلم بنفس الطريقة؟

❌ (في المخ مختلف backprop السلايد وضح إن الـ لا)

3 recurrent structures هل يمثّل؟

❌ مش السبب الأساسي

4 هل من غيره مستحيل ندرّب شبكة؟

❌ شبكة صغيرة آه، كبيرة؟ مستحيل عمليًا

✅ الإجابة الصح:

مش سحر، لكنه Backprop

- الطريقة العملية الوحيدة لتدريب شبكات عميقة وكبيرة

🔥 Jacobian المشكلة الكبيرة: الـ

👉 تعالى نفهم الكارثة

لو:

- حجمهم (x) و (y): $32 \times 32 \times 512$
- $\text{Jacobian} = (D \times D)$

📦 الحجم؟

➡ TB Memory حوالي 1 🤯

مستحيل يتخزن أو يتحسب.

💠 الحل العبقرى

بديل ما نحسب:

[
 $\frac{df}{dx}$
]

نحسب:

[
 $p^T \cdot \frac{df}{dx}$
]

وده

- **Vector–Matrix Product**
- حجمه صغير

- بيعله فعلاً Backprop وده اللي

يعني:

كامل Jacobian إحنا عمرنا ما بنحسب الـ
Projected Jacobian بنحسب

◆ output Scalar؟ لو الـ

لو:

- (y = 1) (Loss scalar)
- (D) حجمه (x)

يبقى:

[
 $\frac{d\mathcal{L}}{dx} \in \mathbb{R}^{D \times 1}$
]

✓ صغير

✓ ممكن

✓ عملي

◆ Chain Rule بشكل بصري

بدل:

Jacobian × Jacobian × Jacobian (مستحيل)

بنعمل:

Vector × Jacobian → Vector

Vector × Jacobian → Vector

الـ الحجم ثابت وصغير طول الطريق

◆ AutoDiff Toolboxes (PyTorch / TensorFlow)

- بتخليك تكتب:

loss.backward()

- وهي:
 - Computational Graph بتبني
 - Backprop بتعمل

- بكفاءة gradients بتحسب

◆ Jacobian طيب لسه محتاجين؟

✓ آه

بس:

- Sparse غالبًا
- مشتقاتهم بسيطة جدًا ReLU / Sigmoid / Tanh
- خاص structure له Softmax

📌 مثال:

- ReLU Jacobian = 0 1 أو
- Softmax Jacobian دقيقًا، بيتحسب بطريقة ذكية

🧠 الخلاصة الذهبية (للامتحان)

- Backprop = Chain Rule as an Algorithm
- يعتمد على:
 - Local gradients
 - Reusing computations
- كامل Jacobian لا يحسب
- Vector-Jacobian products يحسب
- ممكن عمليًا Deep Learning ده اللي خلى

بطريقة مبسطة وملخصة، خطوة خطوة، مع التركيز على أهم النقاط Computational Graphs تمام! خلينا نشرح

1 Forward Graph (Forward Pass)

الفكرة الأساسية:

في الشبكة خطوة بخطوة من البداية للنهاية (module) احنا بنحسب قيم كل وحدة

الخطوات:

1. Compute activation:

- لكل وحدة في الشبكة، احسب الإخراج
- [

$$h = h_w(x)$$

]

(weights) هي المعاملات (w) هو الإدخال و (x) حيث

2. Update input:

- للوحدة التالية ($x := h$) ، نجعل (h) بعد حساب

3. Store intermediate variables:

- **backpropagation** لأنها هتفيدنا لاحقاً في (h) خزن القيم الوسيطة
- هذه الخطوة توفر وقت الحساب لاحقاً، لكنها تستهلك بعض الذاكرة

4. Repeat recursively:

- كرر الخطوات السابقة لكل وحدة في الشبكة بالترتيب الصحيح (من البداية للنهاية)

ملخص:

- حفظ القيم الوسيطة + (activation) حساب كل إخراج = Forward pass

2 Reverse Graph (Reverse Pass / Backpropagation)

الفكرة الأساسية:

(x) ومدخل (w) لكل وزن gradients ، نرجع من النهاية للبداية لحساب forward pass الآن بعد ما حسبنا الـ

الخطوات:

1. Use gradient functions:

- بدلاً من استخدام القيم فقط، نستخدم دوال المشتقة لكل وحدة:
[
 $\frac{\partial h}{\partial x}, \frac{\partial h}{\partial w}$
]

2. Need forward activations:

- gradients لحساب forward pass نحتاج القيم المحفوظة من

3. Sum over mini-batch:

- batch من كل مثال في الـ (gradients) من البيانات، نجمع كل التدرجات batch لو عندنا

4. Reverse-mode automatic differentiation:

- **forward pass** ، لأن اتجاه حساب التدرجات يكون عكس اتجاه البيانات في reverse-mode AD العملية دي اسمها

ملخص:

- Backward pass = استخدام مشتقات كل وحدة + الاستفادة من القيم المحفوظة + حساب

3 Why Save Forward Activations?

- ده هيستهلك وقت كبير $\rightarrow$ backprop ، هنضطر نحسبها من جديد أثناء forward pass لو ما خزّتش القيم الوسيطة في
- نستخدم شوية ذاكرة لتوفير وقت الحساب: trade-off، لذا، نفضل

4 Key Points (النقاط المهمة)

النقطة	الشرح البسيط
Forward Pass	لكل وحدة وتخزينها activations حساب
Reverse Pass	باستخدام مشتقات كل وحدة gradients حساب
Saving activations	، يحتاج ذاكرة backprop ضروري لتسريع الـ
Mini-batch gradients	batch لكل مثال في gradients نجمع
Reverse-mode AD	data flow من النهاية للبداية، عكس gradients حساب

🔦 تخيلها كده:

- تمرير الكرة من البداية للنهاية وحفظ كل مكان الكرة مرت بيه = Forward pass
- إرجاع الكرة من النهاية للبداية واستخدام كل الأماكن اللي سجلناها لحساب تأثير كل خطوة على النتيجة النهائية = Backward pass

بطريقة ملخصة وبسيطة خطوة خطوة مع أمثلة ذهنية **Reverse Graph و Backpropagation** تمام ❤️ هشرحلك النقاط دي كلها عن تساعدك تفهمها للفينال

1 Reverse Graph / Backpropagation – الفكرة العامة

- input (x) وكل (w) لكل وزن **gradients**، نرجع للخلف لحساب (h) activations نحسب كل **Forward Pass** بعد ما نعمل
- **Backpropagation** أو **Reverse-mode automatic differentiation** العملية دي اسمها
- **forward pass** يكون عكس تدفق البيانات في الـ **gradients** لأن تدفق الـ "reverse"؟ ليه اسمها

مثال ذهني:

($x \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow L$) لو عندك سلسلة من المربعات متصلة

- Forward: نحسب (h_1, h_2, h_3) و L
- Backward: ونحسب L نبدأ من ($\frac{dL}{dh_3}, \frac{dL}{dh_2}, \frac{dL}{dh_1}$)

2 Gradient Functions

- **gradient function:** لازم يكون عندها layer كل وحدة أو
 - $\frac{\partial h}{\partial x}$ بالنسبة للـ input
 - $\frac{\partial h}{\partial w}$ بالنسبة للـ weights
- loss لحساب تأثير كل متغير على **Backprop** نستخدمهم في

مثال:

Sigmoid ($\sigma(z)$):

```
[  
  \sigma'(z) = \sigma(z)(1-\sigma(z))  
]
```

بسهولة بدون إعادة الحساب gradient ، نقدر نحسب forward pass من الـ ($\sigma(z)$) لو خزننا

3 Forward Activations ضرورية

- **gradients** لأنها مطلوبة لحساب forward pass من الـ **activations** لازم نخزن
- بدونها، لازم نعيد حساب كل شيء، وده هيكون مكلف وقتياً

4 Mini-batch Gradients

- **weights** من كل العينات قبل ما نحدث الـ **gradients** ، لازم نجمع **mini-batch** لو بنشتغل على
- ده بيضمن إن التحديث يعكس كل البيانات، مش مجرد عينة واحدة

5 Backpropagation – مبسط – خطوات

1. **Forward pass:** احسب كل ($h = h(x)$) لكل layer و **store**ها.
2. **Backward pass:** chain rule ، واستخدم layer ابدأ من آخر
[
 $\frac{dL}{dw} = \frac{dL}{dh} \cdot \frac{dh}{dw}$
]
3. **Update weights:** optimizer أو أي SGD في gradients استخدم

6 مثال رقمي بسيط

- Forward:
[
 $h = \sigma(wx), \quad L = 0.5 (y^* - h)^2$
]

- Backward:
 - [$\frac{dL}{dh} = -(y^* - h)$]
 - [$\frac{dL}{dw} = \frac{dL}{dh} \cdot x \cdot h(1-h)$]
 - forward pass من (h) intermediate ، بس المهم نحفظ (chain rule) كل حاجة بسيطة رياضياً
-

7 Visualization

- graph: الشبكة عبارة عن
 - Nodes = activations أو متغيرات
 - Edges = عمليات رياضية
 - Forward: تنتقل القيم للأمام
 - Backward: chain rule باستخدام edges للخلف عبر نفس الـ gradients تنتقل الـ
-

8 لماذا مهم؟

- loss كمية تأثير وزن معين على gradient كل
 - بكفاءة weights update بدونها ما نقدر نعمل
 - وفعل **gradient computation modular** يخلي **backprop** حتى لو الشبكة كبيرة ومعقدة،
-

9 المفهوم الكبير

- Backprop = **chain rule + smart bookkeeping + efficient computation**
 - **sum gradients + ترتيب العمليات + activations** سهل رياضياً، لكن التنفيذ يحتاج حفظ
-

ملخص بصري ذهني

Forward Pass:

$x \rightarrow h1 \rightarrow h2 \rightarrow h3 \rightarrow L$

(store h1, h2, h3)

Backward Pass:

$dL/dh_3 \rightarrow dL/dh_2 \rightarrow dL/dh_1 \rightarrow dL/dw$

(chain rule, use stored activations)
